16 gennaio 2020



Politecnico di Milano

Anno accademico 2019-2020

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di un dizionario: una struttura dati adatta a memorizzare un insieme finito di parole costruite a partire da un alfabeto.

```
public interface Dictionary  {
  // \result==true sse word è in this
  public /*@ pure @*/ boolean contains(String word);

  // aggiunge word al dizionario
  public void add(String word) throws NullPointerException;

  // \result==true se è presente nel dizionario una parola x della stessa
  // lunghezza di word che differisce al più di un carattere da word stessa
  public /*@ pure @*/ boolean similar(String word);

  // \result è una lista di tutte le parole nel dizionario che iniziano per prefix;
  // se non ve n'è nessuna, \result è  la lista vuota.
  // La stringa prefix deve essere lunga almeno tre caratteri
  public /*@ pure @*/ List<String> complete(String prefix);
}
```

Si supponga che tutte le parole siano costituite esclusivamente dai 26 caratteri minuscoli (a, b, ..., x, y, z).

NB: Per estrarre il carattere (di tipo `char`) in posizione i di una stringa `word`, si può usare la notazione `word.charAt(i)`, con i nell'intervallo `0...word.length()-1`.

Domanda a)

Si specifichino in JML i metodi `add`, `similar`, e `complete`.

Soluzione

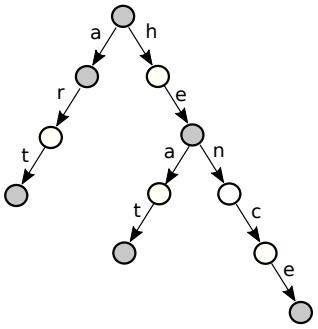
```
//@ensures word!=null && contains(word) &&
//@ (\forall String x; \old(contains(x)); contains(x)) &&
//@ (\forall String x; contains(x); x.equals(word) || \old(contains(x)));
//@signals(NullPointerException e) word == null &&
//@  (forall String x;; \old(contains(x)) <==> contains(x));
public void add(String word) throws NullPointerException;

//@requires word!=null
//@ensures \result <==>
//@       (\exists String x; contains(x) && x.length()==word.length() &&
//@          (\numof int i; 0<=i & i<x.length();
//@                         x.charAt(i)!=word.charAt(i))<=1);
public /*@ pure @*/ boolean similar(String word);

//@requires prefix!=null && prefix.length>=3;
//@ensures (forall String x;;
                  \result.contains(prefix+x) <==> this.contains(prefix+x));
public /*@ pure @*/ List<String> complete(String prefix);
```

Domanda b)

Si consideri la seguente implementazione (parziale) di un dizionario, tramite una struttura dati comunemente nota come Trie. Un trie memorizza ogni parola del dizionario in un albero in modo da consentire una ricerca veloce. Ogni nodo dell'albero può avere fino a 26 figli, uno per ciascuna lettera. Ogni figlio di un nodo corrisponde a un carattere. Le parole sono lette partendo dalla radice e arrivando fino a un nodo marcato come "end-of-word". Tutte le foglie sono "end-of-word", ma anche alcuni nodi intermedi possono esserlo. Un esempio di Trie è mostrato in figura. La radice corrisponde alla parola vuota, che si assume faccia sempre parte del dizionario. In grigio sono raffigurati i nodi "end-of-word". Le parole (non vuote) presenti sono: a, art, he, heat, hence.




```
class Node {
  Node next[] = new Node[26];
  boolean isEndOfWord = false;
}

class Trie implements Dictionary {
  private final Node root;

  // Costruisce un trie con la sola radice (corrispondente alla parola vuota).
  public Trie() {
    root = new Node(); root.isEndOfWord = true;
  }

  // Metodo helper che trova il nodo finale della sequenza che corrisponde alla
  // stringa prefix, se esiste. Il nodo restituito non è necessariamente endOfWord.
  // Restituisce null se prefix non è presente nel trie.
  private /* @ pure helper @ */ Node terminal(String prefix) {
    if (prefix.equals("")) return root;
    Node current = root;
    for (char x : prefix.toLowerCase().toCharArray()) {
      int index = x - 97;
      if (current.next[index] == null) return null;
      current = current.next[index];
    }
    return current;
  }

  public boolean contains(String word) {
    final Node n = terminal(word);
    if (n == null || !n.isEndOfWord) return false;
    return true;
  }

  public void add(String word) throws NullPointerException {
    if (word == null) throw new NullPointerException();
    if (word.equals("")) return;
    Node current = root;
    for (final char x : word.toLowerCase().toCharArray()) {
      int index = x - 97;
      if (current.next[index] == null) current.next[index] = new Node();
      current = current.next[index];
    }
    current.isEndOfWord = true;
  }
}
```

Il metodo privato `terminal` permette di trovare il nodo che corrisponde al prefisso di una parola presente nel dizionario. E' utile per implementare il metodo `contains` e gli altri metodi di `Dictionary`, che per semplicità non riportiamo.

Scrivere un invariante di rappresentazione per la classe `Trie`. Si suggerisce di sfruttare il metodo privato (puro) `terminal`.

Soluzione

L'invariante deve stabilire che la struttura dati è un albero, ossia che non vi sono stringhe differenti che terminano nello stesso nodo e che `isEndOfWord` è vero per tutte le foglie.

```
private invariant
   root != null && root.isEndOfWord &&
   //non ci sono stringhe diverse che terminano nello stesso nodo
   (\forall String x; terminal(x)!=null;
       (\forall String y, terminal(y)!=null;
                !x.equals(y) ==> !terminal(x).equals(terminal(y)))) &&
   // isEndOfWord è vero per tutte le foglie:
   (\forall String x; terminal(x)!=null;
        (\forall int i; 0<=i && i<=25; terminal(x).next(i)==null) ==>
        terminal(x).isEndOfWord);
```

Esercizio 2

Si consideri la seguente classe Java:

```
public class Account {
  private static List<Double> log = new ArrayList<Double>();
  private double balance;

  public Account() { balance = 0.0; }

  public synchronized void deposit(double val) {
    balance = balance+val;
    log.add(val);
  }

  public synchronized void withdraw(double val) {
    balance = balance-val;
    log.add(-val);
  }
}
```

Domanda a)

La sincronizzazione è corretta? In caso affermativo motivare la risposta, in caso negativo indicare come andrebbe modificato il codice per avere una sincronizzazione corretta.

Soluzione

Avere sincronizzato i due metodi `deposit` e `withdraw` non è sufficiente perchè l'attributo `log` è statico, quindi condiviso da tutte le istanze della classe. Occorre sincronizzare l'accesso all'attributo `log` modificando la linea:
`log.add(val)`
in:
`synchronized(log)  log.add(val);`
e la linea:
`log.add(-val)`
in:
`synchronized(log)  log.add(-val);`

Domanda b)

Modificare la classe affinchè il chiamante del metodo `withdraw` venga sospeso se mancano fondi per il prelievo.

Soluzione

```
public class Account {
  private static List<Double> log = new ArrayList<Double>();
  private double balance;

  public Account() { balance = 0.0; }

  public synchronized void deposit(double val) {
    balance = balance+val;
    synchronized(log) { log.add(val); }
    notifyAll();
  }

  public synchronized void withdraw(double val) {
    while(balance<val) try { wait(); } catch(Exception ex) { ex.printStackTrace(); }
    balance = balance-val;
    synchronized(log) { log.add(-val); }
  }
}
```

Domanda c)

Si aggiunga alla classe un metodo `void printLog()` che stampa l'intero `log` in maniera asincrona rispetto al chiamante.

Soluzione

```
public void printLog() {
  List<Double> logCopy;
  synchronized(log) {
    logCopy = new ArrayList<Double>(log);
  }
  new Thread(() -> logCopy.forEach(System.out::println)).start();
}
```

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
public class Customer { ... }
public class RegisteredCustomer extends Customer { ... }

public class Shop {
   public int computeDiscount(Customer c) { return 0; }
}
public class OnlineShop extends Shop {
   public int computeDiscount(Customer c) { return 5; }
   public int computeDiscount(RegisteredCustomer c) { return 15; }
}
public class OnlinePremiumShop extends OnlineShop {
```

```
    public int computeDiscount(RegisteredCustomer c) {
        return 20 + super.computeDiscount(c);
    }
}
```

Si consideri poi il seguente frammento di codice Java che utilizza le classi dichiarate sopra.

```
01. Customer c1 = new Customer();
02. Customer c2 = new RegisteredCustomer();
03. RegisteredCustomer c3 = new RegisteredCustomer();
04. Shop s1 = new Shop();
05. Shop s2 = new OnlinePremiumShop();
06. OnlineShop s3 = new OnlinePremiumShop();
07. OnlinePremiumShop s4 = new OnlineShop();
08. OnlineShop s5 = new OnlineShop();
09. System.out.println(s1.computeDiscount(c1));
10. System.out.println(s1.computeDiscount(c2));
11. System.out.println(s2.computeDiscount(c2));
12. System.out.println(s2.computeDiscount(c3));
13. System.out.println(s3.computeDiscount(c2));
14. System.out.println(s3.computeDiscount(c3));
15. System.out.println(s4.computeDiscount(c1));
16. System.out.println(s4.computeDiscount(c2));
17. System.out.println(s5.computeDiscount(c1));
18. System.out.println(s5.computeDiscount(c3));
19. s2 = s1;
20. s3 = s1;
21. s5 = s2;
22. s3 = s5;
```

Domanda a)

Si indichino quali righe del codice sopra riportato generano un errore in fase di compilazione, motivando brevemente la propria risposta.

Soluzione

- La riga 7 non compila in quanto `OnlineShop` non è sottotipo di `OnlinePremiumShop`. Di conseguenza non compilano le righe che utilizzano `s4`, ovvero le righe 15 e 16.
- La riga 20 non compila in quanto il tipo statico di `s1` non è sottotipo del tipo statico di `s3`.
- La riga 21 non compila in quanto il tipo statico di `s2` non è sottotipo del tipo statico di `s5`.

Domanda b)

Supponendo che tutte le righe che provocano errori siano state rimosse, scrivere l'output prodotto dal programma, motivando brevemente la risposta.

Soluzione

```
0  // Chiama il metodo con parametro Customer in Shop
0  // Chiama il metodo con parametro Customer in Shop
5  // Chiama il metodo con parametro Customer in OnlineShop
5  // Chiama il metodo con parametro Customer in OnlineShop
5  // Chiama il metodo con parametro Customer in OnlineShop
```

```
35 // Chiama il metodo con parametro RegisteredCustomer in OnlinePremiumShop
5  // Chiama il metodo con parametro Customer in OnlineShop
15 // Chiama il metodo con parametro RegisteredCustomer in OnlineShop
```

Esercizio 4

Si consideri il seguente frammento di codice:

```
int f(int a, int b) {
   if (a > b) {
      System.out.println("A");
   } else {
      System.out.println("B");
   }
   for (int i=0; i<1000; i++) {
      if (a > 0) break;
      else a = 1;
   }
}
```

Domanda a)

Definire un insieme minimo di casi di test per coprire tutte le istruzioni (statement coverage)

Soluzione

Servono due casi di test: `[a=-1, b=1], [a=2, b=1]`

Domanda b)

Definire un insieme minimo di casi di test per coprire tutte le decisioni (edge coverage).

Soluzione

Non è possibile coprire tutte le decisioni, in quanto la condizione `i<1000` non viene mai valutata come `false`. Per valutare tutte le dicisioni che possono essere coperte, i casi di test precedenti sono sufficienti.

Domanda c)

Definire un insieme minimo di casi di test per coprire tutti i cammini (path coverage).

Soluzione

L'esecuzione passa necessariamente dal ramo `if` oppure dal ramo `else`, ed entra nel ciclo `for` una oppure due volte. I possibili cammini sono dunque 4: uno che entra nel ramo `if` ed esegue il ciclo una volta, uno che entra nel ramo `if` ed esegue il ciclo due volte, uno che entra nel ramo `else` ed esegue il ciclo una volta, uno che entra nel ramo `else` ed esegue il ciclo due volte.

Servono quindi 4 casi di test: `[a=1, b=0], [a=1, b=2], [a=-1, b=0], [a=-1, b=-2]`